

CMCMIS_SIMPLIFIED

PHASE 10

REPORTS & ANALYTICS

FULLY SEALED · END-TO-END TECHNICAL DOCUMENTATION

DS LOCKED SEAL

This document records the Phase 10 implementation as a senior engineering build record: database thinking, backend architecture, frontend UI logic, RBAC, reporting exports, analytics charts, and verification closure. It intentionally avoids full code dumps; it documents the logic, flow, decisions, operational reasoning, and quality gates needed for future maintenance.

Field	Value
Document name	phase10ReportsAnalyticsFULLYSEALEDsealpacked.docx
Project	CMCMIS_SIMPLIFIED
Phase	Phase 10 — Reports & Analytics
Status	SEALED · 41/41 smoke green
Audience	DS · future self · senior reviewers · project handover
Engineering posture	Read-only reporting, no destructive schema changes, government-grade RBAC
Primary outcome	6 reports + 8 analytics charts + PDF/CSV export + permission-aware UI

Document Seal and Reading Contract

This document is written as the Phase 10 sealed build record. It is intentionally long, step-by-step, and review-friendly. The goal is not only to remember what was built, but also to preserve why each database, backend, PDF, analytics, and frontend decision was made.

HOW TO READ THIS DOCUMENT

Read Part I for the executive view. Read Parts III to VI when coding or debugging. Read Part VII and VIII for report/chart-specific behavior. Read Part IX and X for RBAC and smoke verification. Read appendices when doing handover or audits.

Phase 10 is not a mock reporting page. It is a live reporting and analytics surface over the sealed CMCMIS database. The module reads legacy and MVP tables using repository aliasing, applies role-permission checks at route level, and streams exports without storing generated files.

Layer	Locked expectation
Database	Read-only operational queries; no Phase 10 table ALTERs; no report logs; no persisted PDFs.
Backend	Routes -> controllers -> services -> repositories; validators at entry; PDF/CSV streaming.
Frontend	Permission-aware report cards, filters, summary tiles, TanStack tables, charts, and export actions.
Security	JSON routes require report view permission; PDF/CSV export additionally requires reports:export.
Verification	Smoke matrix reports 41 passed, 0 failed with JSON, PDF, CSV, RBAC, pagination, and cross-truth checks.

Table of Contents — Senior Engineering Map

Part	Title	What it gives you
I	Executive Overview	Mission, final state, numbers, and doctrine
II	Phase Context	How Phase 10 connects to Phases 0-9
III	Decision Register	P10-D1 through P10-D8
IV	Database & Backend Thinking	Live DB introspection, SQL logic, pagination, derived fields
V	Backend Build Record	Reports module, analytics module, PDF, CSV, routes, server mount
VI	Frontend Build Record	Reports page, filters, tables, charts, hooks, permission UI
VII	Six Reports Deep Dive	R1 through R6
VIII	Eight Analytics Deep Dive	G1 through G8
IX	RBAC, Security, and Export Control	Permission matrix and gate layering
X	Verification and Smoke Matrix	41/41 evidence, cross-truth, build verification
XI	Operational Readiness	How to use, monitor, debug, and hand over
XII	Appendices	Endpoint matrix, file inventory, glossary, checklists

Page numbers are intentionally not manually listed because Word/LibreOffice pagination can change after edits. Every part starts on its own page and is labeled clearly for navigation.

1. Phase 10 Mission Statement

Phase 10 converts CMCMS from operational workflow software into a measurable management system. Before this phase, the application could authenticate users, manage equipment, create job requests, convert requests into job cards, and execute job cards. After this phase, managers and engineers can inspect live operational truth through reports, PDFs, and analytics charts.

The module is intentionally read-only. It does not create report history tables, it does not store generated PDFs, and it does not mutate calibration/job data. That is the correct architecture for a first production-grade reporting slice: reports observe the system; they do not become another source of truth.

The result is six operational reports, eight analytics datasets, PDF export, CSV export, filter controls, pagination, RBAC-gated route access, and a verified smoke matrix.

LOCKED OUTCOME

6 reports + 8 analytics charts + PDF/CSV export + RBAC + clean FE build + 41/41 smoke pass. The module is sealed as a production reporting surface, not a prototype dashboard.

2. Final State Snapshot

The final Phase 10 state can be understood as a three-layer report engine. The database layer provides live data through real MySQL tables. The backend layer shapes, filters, validates, and exports that data. The frontend layer provides a clean report selection, filter bar, summary tiles, detailed table, analytics grid, and export panel.

The most important architectural win is that the frontend never guesses legacy column names or database status rules. It receives canonical payloads from the backend and renders them. The repository layer is the translation boundary.

Metric	Value
Report JSON endpoints	6
Report PDF endpoints	6
Analytics JSON endpoints	8
Analytics CSV endpoints	8
Total endpoints	28
New report permissions	8
Backend files delivered	15
Frontend files delivered	13+ root patches
Smoke matrix	41 passed / 0 failed
Frontend build	2513 modules transformed, clean

3. The One-Line Architecture

CMCMIS Phase 10 is a read-only Reports & Analytics layer over the sealed MySQL schema, with backend repository aliasing, service-level summaries, PDFKit streaming, hand-rolled CSV export, permission-aware frontend cards, TanStack report tables, and Recharts analytics panels.

In one mental picture: user selects report -> filter payload validated -> repo queries live tables -> service builds summary + rows -> frontend renders table/charts -> export streams PDF/CSV through a second permission gate.

SENIOR ENGINEER MENTAL MODEL

A report is not a page. A report is a contract: filters in, canonical data out, summary consistent with rows, export consistent with JSON, and RBAC identical across UI and backend.

4. What Phase 10 Deliberately Did Not Do

The phase avoided false complexity. It did not add report tables, report logs, scheduled report generation, background queues, Redis, or stored PDF blobs. These choices match the sealed system constraints: MySQL is enough, generated PDFs are streamed only, and file storage is explicitly out of scope until specified by the organization.

This discipline matters because reporting modules often become hidden write paths. Phase 10 refuses that pattern. It reads, computes, renders, and streams.

- No table ALTERs for report logic.
- No persisted generated PDFs.
- No SheetJS dependency.
- No report history table.
- No mock analytics data.
- No FE-side role truth. Backend enforces permissions independently.

5. Phase 10 Delivery Summary

The work delivered both operational reports and managerial analytics. Reports are table-and-PDF oriented. Analytics are chart-and-CSV oriented. They share the same database reality, filter discipline, permission model, and export ideology.

This split is important. Reports answer: "What exact rows should I inspect or print?" Analytics answer: "What patterns are emerging across time, status, division, engineer, and job type?"

Surface	Purpose	Export
Reports R1-R6	Operational row-level reports with summaries and tables	PDF
Analytics G1-G8	Aggregated chart datasets for trends and distributions	CSV
Frontend /reports	Unified user-facing workspace	PDF, CSV, print
Backend /reports	Validated report JSON + PDF streaming	JSON, PDF
Backend /analytics	Validated chart JSON + CSV streaming	JSON, CSV

6. Phase 10 in the Phase Roadmap

Phase 10 is positioned after the critical transaction modules. That order is correct. A reporting system is only trustworthy after the underlying operational records are created, assigned, executed, completed, verified, and scoped correctly.

Phases 6 to 9 supplied the data engine: job requests, job cards, conversion, task execution, documents, closure, and hotfixes. Phase 10 uses those records to expose operational insight.

Previous phase	What it made available for Phase 10
Phase 6	Job Request and Job Card list foundations, canonical mapping, row-level scope
Phase 7 Slice 2	Conversion flow, approval/assignment linkage, auto-created Job Cards
Phase 8	Dashboard and Inquiry patterns, KPI thinking, search/index discipline
Phase 9	Job Card detail execution, task/doc/closure fields, completion states
Phase 9 hotfix	Maintenance/Spares CRUD, date input correctness, Mark Complete/Closure gate fixes

7. Why Reports Could Not Come Earlier

A report must reflect real lifecycle semantics. If reports were built before job requests, conversion, job cards, task completion, and closure were stable, the reports would have encoded premature assumptions.

Phase 10 benefits from earlier sealed doctrines: repository aliasing, permission middleware, zod validation, employee_id identity, no destructive schema edits, PDF streaming, and frontend feature-based folders.

DOCTRINE CONTINUITY

Phase 10 does not invent a new architecture. It extends the locked architecture: Routes -> Controllers -> Services -> Repositories on the backend and feature-based pages/hooks/api modules on the frontend.

8. Relationship to FINAL-DESC Requirements

The final project description listed reports, analytics, and exports as important post-core capabilities. Phase 10 implements that capability without violating the project constraints: on-prem friendly, no file storage for generated PDFs, MySQL-based queries, and desktop-first professional UI.

The implementation also respects the earlier decision ledger: JavaScript only, raw SQL via mysql2/promise, React Query for server state, PDFKit over Puppeteer, and streamed PDFs.

- Reports are generated from live MySQL data.
- PDFs stream and are never stored as records.
- Frontend uses React Query for report/chart server state.
- Backend remains layered and feature-module based.
- RBAC is permission-code based, not role-string shortcuts.

9. How to Think About Reporting Logic

Reporting logic has two axes: row truth and aggregate truth. Row truth means the table rows match the filters. Aggregate truth means the summary tiles and chart totals match the same data definition. Phase 10 keeps both through repository-level SQL and service-level payload assembly.

Do not think of each report as a custom page. Think of a reusable pipeline where each report supplies: permission code, endpoint key, filter schema, list query, summary query, columns, PDF title, and frontend card metadata.

LOGIC DEVELOPMENT PATTERN

Define the business question first. Then define the canonical columns. Then map them to real DB columns. Then define filters. Then write list and count/summary queries. Then wire the service response. Only after that should the UI be built.

10. The Phase 10 Senior Engineering Checklist

A senior report module must be correct, safe, explainable, and verifiable. Phase 10 follows that shape. The implementation is not just about making charts appear; it is about preserving trust in every number shown to a government-grade technical user.

- Can the report be generated without modifying source data?
- Can every field be traced back to a real table/column or derived rule?
- Can a user without export permission view but not download?
- Can View-Only users read permitted reports without receiving export capability?
- Do PDF and JSON endpoints share the same dataset semantics?
- Do charts export the same data they display?
- Do smoke tests prove both access control and data correctness?

11. Decision P10-D1 — Persistence

P10-D1 locks a critical implementation choice for Phase 10. The decision is: No report logs, no persisted PDFs. Every report read fetches live data and PDF streams to the response.

This decision protects the module from one of the common reporting-system failures: hidden writes, schema leakage, stale derived values, inconsistent export security, oversized dependencies, spreadsheet encoding problems, reserved-word bugs, or mysql2 result-shape mistakes.

Future engineers should treat this decision as a local law unless a later sealed phase explicitly revises it.

DECISION LOCKED

No report logs, no persisted PDFs. Every report read fetches live data and PDF streams to the response.

12. Decision P10-D2 — Column aliasing

P10-D2 locks a critical implementation choice for Phase 10. The decision is: Repositories alias legacy EQM/JR/JM columns into canonical snake_case. Service, controller, and frontend never consume legacy names.

This decision protects the module from one of the common reporting-system failures: hidden writes, schema leakage, stale derived values, inconsistent export security, oversized dependencies, spreadsheet encoding problems, reserved-word bugs, or mysql2 result-shape mistakes.

Future engineers should treat this decision as a local law unless a later sealed phase explicitly revises it.

DECISION LOCKED

Repositories alias legacy EQM/JR/JM columns into canonical snake_case. Service, controller, and frontend never consume legacy names.

13. Decision P10-D3 — last_cal_date derivation

P10-D3 locks a critical implementation choice for Phase 10. The decision is: No EQM_CAL_DATE exists. Last calibration date is derived from the most recent verified-closed calibration job card.

This decision protects the module from one of the common reporting-system failures: hidden writes, schema leakage, stale derived values, inconsistent export security, oversized dependencies, spreadsheet encoding problems, reserved-word bugs, or mysql2 result-shape mistakes.

Future engineers should treat this decision as a local law unless a later sealed phase explicitly revises it.

DECISION LOCKED

No EQM_CAL_DATE exists. Last calibration date is derived from the most recent verified-closed calibration job card.

14. Decision P10-D4 — RBAC split

P10-D4 locks a critical implementation choice for Phase 10. The decision is: JSON requires the report view permission; PDF/CSV export additionally requires reports:export.

This decision protects the module from one of the common reporting-system failures: hidden writes, schema leakage, stale derived values, inconsistent export security, oversized dependencies, spreadsheet encoding problems, reserved-word bugs, or mysql2 result-shape mistakes.

Future engineers should treat this decision as a local law unless a later sealed phase explicitly revises it.

DECISION LOCKED

JSON requires the report view permission; PDF/CSV export additionally requires reports:export.

15. Decision P10-D5 — PDF library

P10-D5 locks a critical implementation choice for Phase 10. The decision is: PDFKit is used. PDFs stream to res; no browser/Chromium dependency; bufferPages enables Page X of Y footers.

This decision protects the module from one of the common reporting-system failures: hidden writes, schema leakage, stale derived values, inconsistent export security, oversized dependencies, spreadsheet encoding problems, reserved-word bugs, or mysql2 result-shape mistakes.

Future engineers should treat this decision as a local law unless a later sealed phase explicitly revises it.

DECISION LOCKED

PDFKit is used. PDFs stream to res; no browser/Chromium dependency; bufferPages enables Page X of Y footers.

16. Decision P10-D6 — CSV serializer

P10-D6 locks a critical implementation choice for Phase 10. The decision is: CSV is hand-rolled with UTF-8 BOM so Excel opens it correctly. No SheetJS dependency.

This decision protects the module from one of the common reporting-system failures: hidden writes, schema leakage, stale derived values, inconsistent export security, oversized dependencies, spreadsheet encoding problems, reserved-word bugs, or mysql2 result-shape mistakes.

Future engineers should treat this decision as a local law unless a later sealed phase explicitly revises it.

DECISION LOCKED

CSV is hand-rolled with UTF-8 BOM so Excel opens it correctly. No SheetJS dependency.

17. Decision P10-D7 — delayed alias

P10-D7 locks a critical implementation choice for Phase 10. The decision is: MariaDB treats delayed as reserved; SQL returns delayed_count and service maps to delayed.

This decision protects the module from one of the common reporting-system failures: hidden writes, schema leakage, stale derived values, inconsistent export security, oversized dependencies, spreadsheet encoding problems, reserved-word bugs, or mysql2 result-shape mistakes.

Future engineers should treat this decision as a local law unless a later sealed phase explicitly revises it.

DECISION LOCKED

MariaDB treats delayed as reserved; SQL returns delayed_count and service maps to delayed.

18. Decision P10-D8 — mysql2 destructuring

P10-D8 locks a critical implementation choice for Phase 10. The decision is: Use `const [rows]` for direct queries and `const [[rows]]` for `Promise.all` query tuples.

This decision protects the module from one of the common reporting-system failures: hidden writes, schema leakage, stale derived values, inconsistent export security, oversized dependencies, spreadsheet encoding problems, reserved-word bugs, or `mysql2` result-shape mistakes.

Future engineers should treat this decision as a local law unless a later sealed phase explicitly revises it.

DECISION LOCKED

Use `const [rows]` for direct queries and `const [[rows]]` for `Promise.all` query tuples.

19. Step 0 — Read-Only Introspection Before Code

Phase 10 began correctly: by reading the project structure and verifying real database columns before writing SQL. This is the right order for a legacy-backed system. A report query written from memory is dangerous because one wrong column assumption can silently produce empty or misleading reports.

The introspection confirmed the existing backend feature-module structure, middleware stack, repository pattern, and relevant legacy table names.

- Backend modules already included auth, dashboard, employees, equipment, inquiry, jobCards, jobRequests, lookups, users.
- Middleware stack already included authenticate, authorize, rowLevelScope, validate, and errorHandler.
- Reports and analytics were added as new modules, not as code inside dashboard or inquiry.

20. Real Database Ground Truth

The most important database lesson was simple: trust the live schema, not the screenshot or mock field name. Phase 10 confirmed actual equipment, section, employee, job request, and job card columns before designing report queries.

This prevents fatal report mistakes such as querying EQM_CAL_DATE when no such column exists or treating employee identity as numeric user_id when the production schema uses employee_id strings.

Concept	Ground truth used in Phase 10
Equipment master	cmms_eqip_mst with EQM_TYPE + EQM_ID composite identity
Equipment name	EQM_NAME
Serial number	EQM_SRNO
Equipment status	EQM_MVP_STATUS enum with ACTIVE, UNDER_CALIBRATION, UNDER_REPAIR, etc.
Next calibration due	EQM_CAL_DUE_DATE
Division	cmms_section_mst using SM_ID, SM_SHORTNAME, SM_NAME
Employee	cmms_emp_mst using EMM_ID and EMM_NAME
Job request status	JR_MVP_STATUS enum with DRAFT, SUBMITTED, ASSIGNED, IN_PROGRESS, COMPLETED, VERIFIED_CLOSED, REJECTED, REOPENED
Job card status	JM_MVP_STATUS enum with ASSIGNED, IN_PROGRESS, COMPLETED, VERIFIED_CLOSED, REOPENED

21. Why No Table ALTERs Were Needed

The report requirements could be satisfied by existing legacy and MVP tables. That meant the correct Phase 10 move was not schema mutation; it was query design. The only migration added permission codes and grants.

This is a strong senior-engineering choice because it reduces deployment risk. Reporting should not lock or rewrite large operational tables unless absolutely necessary.

DATABASE DISCIPLINE

Phase 10 adds no report tables and no business-data columns. Migration 400 seeds permissions only. Operational tables remain untouched.

22. Canonical Column Aliasing Rule

The repository owns database vocabulary. That means it is acceptable for reports.repo.js to know EQM_NAME, JR_MVP_STATUS, and JM_SectionJobNo. It is not acceptable for controllers, services, or frontend components to depend on those names.

This rule is especially important in CMCMIS because the legacy schema has mixed naming, typos, prefixed columns, and composite keys. Repositories translate the mess into clean canonical payloads.

- Repo input: validated filter params.
- Repo output: canonical rows with readable names.
- Service responsibility: attach summaries, scope, labels, export metadata.
- Frontend responsibility: render canonical fields only.

23. last_cal_date Derived Correctly

The schema does not contain a direct EQM_CAL_DATE column. Phase 10 therefore derives last_cal_date from historical job-card evidence instead of inventing a column. The correct source is the latest verified-closed calibration-related job card for that equipment.

This is better than a fake column or a stale cached value. It links the reported date to completed work, which is the only trustworthy source for calibration history.

DERIVATION RULE

last_cal_date = MAX(JM_VERIFIED_ON) where the job card is VERIFIED_CLOSED for the same equipment and the workflow/job type indicates calibration.

24. Filter Logic and Date Windows

Reports use date filters as inclusive whole-day windows. This is important for user trust: if a manager selects 2026-05-20 to 2026-05-20, they expect the whole day, not exactly midnight.

The backend converts dateFrom to start-of-day and dateTo to end-of-day. The query layer then applies those boundaries against report-specific timestamps.

Filter	Backend meaning
dateFrom	Inclusive lower bound at 00:00:00.000000
dateTo	Inclusive upper bound at 23:59:59.999999
divisionId	Filters via cmms_section_mst / equipment division relationship
status	Report-specific allow-listed enum, never interpolated freely
page/page_size	Offset pagination with deterministic sort and count

25. Pagination and Count Strategy

The report tables can be large: job cards and job requests have tens of thousands of rows. Phase 10 therefore treats pagination as mandatory, not optional. Every list query is paired with a count query so the frontend can render page controls honestly.

The report table does not fetch all rows into the browser. This protects memory, network usage, and UI responsiveness.

PERFORMANCE RULE

A report list endpoint must return rows, total, page, and page_size. It should never return 20k+ rows merely because a PDF export exists.

26. Summary Query Design

Every report needs summary tiles. Summary values should not be computed by looping through only the current page. They must represent the whole filtered dataset. Phase 10 solves this by using SQL aggregation over the same filter semantics used by the list query.

The service assembles rows and summaries into one canonical payload so the UI renders the table and tiles from one report contract.

- List query: current page of details.
- Count query: total rows matching filters.
- Summary query: aggregate totals across the whole filtered dataset.
- Service payload: rows + page metadata + summary tiles + report metadata.

27. Why SQL Aggregation Beats Node Aggregation

The MySQL engine is the correct place for counts, grouping, date bucketing, and status distribution. Pulling thousands of rows into Node just to count them is slower, memory-heavy, and harder to verify.

Phase 10 keeps aggregation in repository SQL and keeps Node focused on payload assembly, security decisions, PDF formatting, CSV formatting, and response streaming.

SENIOR RULE

Do not aggregate huge operational report datasets in Node when SQL can do it with indexes and group-by execution plans.

28. Row-Level and Role-Level Scope Thinking

Phase 10 uses permissions as the first line of security. The route determines whether a user can see a report at all. The service can then apply role-aware scope where needed, especially for report types that should be personalized for Normal Users or Lab Engineers.

This mirrors earlier CMCMIS doctrine: permissions grant access to a capability, and row-level logic narrows the records inside that capability.

Layer	Responsibility
Frontend	Hide inaccessible report cards and export buttons.
Route middleware	Reject missing permission with 403.
Service	Apply actor-aware scope and build canonical payload.
Repository	Apply SQL filters with bound params only.
Smoke tests	Prove 401, 403, and 200 behaviors.

29. Backend Dependency Choice

Phase 10 added pdfkit on the backend. This matches the locked architectural decision to use PDFKit over Puppeteer. The system remains on-prem friendly because it avoids a Chromium dependency and can stream generated PDFs directly.

Frontend dependencies were also added, but backend PDF generation remains independent from frontend HTML rendering. Reports are server-generated, not browser screenshots.

Dependency	Used for	Why
pdfkit	PDF report generation	Server-side, on-prem friendly, no browser engine required
mysql2/promise	SQL execution	Already locked; supports prepared statements and pool use
zod	Query validation	Rejects bad filters before repo execution

30. Migration 400 — Permission Seed

Migration 400 is additive and idempotent. It inserts eight reports:* permission codes and role grants. It does not alter operational tables.

This migration is the security spine of Phase 10 because every report and export route depends on these permission codes.

Permission	SA	LIC	LE	NU	VO
reports:view-calibration-due	Y	Y	Y		Y
reports:view-pending-jobs	Y	Y		Y	Y
reports:view-equipment-utilization	Y	Y			Y
reports:view-engineer-summary	Y	Y	Y		Y
reports:view-job-card-summary	Y	Y	Y		Y
reports:view-job-request-summary	Y	Y		Y	Y
reports:view-analytics	Y	Y	Y	Y	Y
reports:export	Y	Y	Y	Y	

31. Reports Module Structure

The reports module follows the sealed backend pattern. Validators define allowed filter shapes. The repository owns SQL. The service assembles business payloads and actor-aware scope. The PDF module renders exports. The controller stays thin. The routes file wires permissions and validators.

This separation makes the module testable and maintainable. A bug in PDF formatting does not affect SQL. A filter validation change does not require frontend rewrites if the API contract stays stable.

File	Responsibility
reports.validators.js	Six report query schemas and enum allow-lists.
reports.repo.js	List and summary SQL for R1-R6.
reports.service.js	Payload assembly, scope, summary, export dataset preparation.
reports.pdf.js	Shared PDF template, report sections, pagination, footer stamping.
reports.controller.js	HTTP handlers for JSON and PDF routes.
reports.routes.js	12 routes with view and export gates.

32. Zod Validators for Reports

Validators are not decoration. They are the first database safety barrier. Phase 10 uses report-specific query schemas to reject unknown parameters, malformed dates, invalid statuses, huge page sizes, and untrusted enum values.

This matters because report endpoints often attract exploratory filter combinations. Strict schemas keep the route contract explicit and protect SQL builders from accidental surface growth.

- `dateFrom/dateTo` must be YYYY-MM-DD.
- `divisionId` must be positive integer.
- `status` is allow-listed per report.
- `page` defaults to 1.
- `page_size` defaults to 50 and is capped at 500.
- Unknown query params are rejected through strict schemas.

33. reports.repo.js — SQL Responsibility

reports.repo.js is the SQL-only file for the six reports. It contains list queries, count queries, and summary queries. It aliases legacy column names to canonical output names.

The repo layer must stay boring and disciplined: no request objects, no response objects, no role branching except where passed as explicit scope, and no PDF formatting.

REPOSITORY CONTRACT

Input: validated params and explicit scope. Output: canonical rows and totals. Never output legacy names. Never mutate data.

34. reports.service.js — Payload Assembly

The service is where database results become report payloads. It chooses the correct repo function, attaches summary tiles, applies actor scope, and shapes the final JSON that the frontend consumes.

The service is also the correct place to prevent role leakage. The frontend may hide a card, but the service still receives an actor and must produce only what that actor is allowed to see.

Service responsibility	Why it belongs in service
Report dispatch	Keeps controllers thin and route files readable.
Summary assembly	Combines list rows and aggregate metrics.
Actor-aware scope	Business rule, not SQL boilerplate.
PDF data preparation	PDF should consume same report semantics as JSON.

35. reports.pdf.js — PDF Engine

The PDF engine is a shared rendering layer for all six reports. It uses PDFKit with A4 landscape layout, header branding, sections, table rendering, auto-pagination, and footer page stamping.

The engine is intentionally server-side and template-driven. It is not a screenshot of the web page. That makes exported PDFs stable, repeatable, and independent of browser layout.

PDF RULE

PDF endpoints require both the report view permission and reports:export. View-Only can view allowed report JSON but cannot export PDF.

36. PDF Auto-Pagination Logic

Large reports such as Pending Jobs, Job Card Summary, and Job Request Summary can produce multi-megabyte PDFs. Phase 10 therefore treats pagination as a core PDF concern.

The renderer tracks row height, available page height, section headers, and footer stamping. PDFKit bufferPages allows the final page count to be known before adding Page X of Y footers.

- A4 landscape gives tables room for operational columns.
- Header contains report identity and organization context.
- Summary section appears before detailed rows.
- Rows continue across pages cleanly.
- Footers are stamped after total page count is known.

37. reports.controller.js — Thin HTTP Layer

Controllers are deliberately thin. They extract actor/context, call service methods, and return JSON or stream PDF. They do not write SQL, compute charts, or branch on database details.

This follows the same discipline used in the rest of CMCMIS: controllers are adapters, not business modules.

CONTROLLER STANDARD

One handler should read request context, call one service function, and shape the response envelope. If it needs complex logic, that logic belongs in service.

38. reports.routes.js — Composite Gate for Export

The route layer wires authenticate, authorize, validate, and controller. For PDF endpoints, Phase 10 implements a composite gate: the user must hold the report-specific view permission and reports:export.

This is the correct split because viewing a report and downloading it are not the same capability. View-Only should be able to inspect but not export.

Route family	Gate
GET /api/v1/reports/:key	Report-specific view permission
GET /api/v1/reports/:key/pdf	Report-specific view permission + reports:export
GET /api/v1/analytics/:key	reports:view-analytics
GET /api/v1/analytics/:key/csv	reports:view-analytics + reports:export

39. Analytics Module Structure

Analytics is intentionally separate from reports. Reports are operational rows; analytics are aggregate datasets for charts. The separate module avoids mixing table export logic with chart aggregation logic.

The module mirrors the report module: validators, repo, service, controller, routes. It adds CSV serialization in service so each chart can export its displayed dataset.

File	Responsibility
analytics.validators.js	Common chart query schema for date/division filters.
analytics.repo.js	Eight aggregation queries G1-G8.
analytics.service.js	JSON payload and CSV serializer.
analytics.controller.js	16 handlers: JSON + CSV for each chart.
analytics.routes.js	16 routes with analytics/export gates.

40. analytics.repo.js — G1 to G8 SQL

Each analytics chart is backed by an SQL aggregation query. That means the chart data is live, filterable, and exportable. The repo uses group-by, date bucketing, status labels, and division/engineer aggregation as needed.

The key design is that chart data and CSV export use the same service path. The user downloads the data they saw, not a separately computed dataset.

ANALYTICS CONTRACT

Chart JSON and chart CSV must share one data definition. If the chart says 12, the exported CSV must explain the same 12.

41. CSV Export Logic

Phase 10 intentionally avoids SheetJS. CSV is sufficient for chart datasets and lighter for on-prem deployment. The CSV serializer adds a UTF-8 BOM so Excel opens the file correctly without breaking non-ASCII text.

Each analytics chart has its own CSV endpoint. This keeps export behavior simple, transparent, and easy to test.

- No SheetJS dependency.
- UTF-8 BOM for Excel friendliness.
- Headers generated from canonical keys.
- Values safely quoted when needed.
- CSV endpoints gated by reports:view-analytics + reports:export.

42. Server Mounting

Phase 10 mounts two new route trees: `/api/v1/reports` and `/api/v1/analytics`. The mount position follows the existing `server.js` pattern: after established modules and before the 404 handler.

This ensures requests flow through the locked middleware pipeline and unmatched routes still receive the standard error envelope.

Mount	Purpose
<code>/api/v1/reports</code>	Six report JSON endpoints and six PDF endpoints.
<code>/api/v1/analytics</code>	Eight chart JSON endpoints and eight CSV endpoints.

43. Backend Error and Response Contract

The reporting endpoints follow existing envelope discipline. Successful JSON responses return a data object. Validation failures return the standard Zod error envelope. Permission failures return 403. Anonymous access returns 401.

This is essential because the frontend can build one error handling pattern for all reports and analytics, rather than special-casing each report.

- 401 for unauthenticated requests.
- 403 for missing permission or missing export permission.
- 422 for invalid filters.
- 200 with data for valid authorized report/chart requests.
- PDF/CSV streamed with correct content disposition and type.

44. Backend Build Verification

The Phase 10 backend was not considered complete after files compiled. It was verified through runtime smoke tests, route access checks, PDF stream generation, CSV export checks, and cross-truth database comparisons.

The strongest evidence is the 41/41 smoke matrix, including anonymous 401s, View-Only export 403s, report JSON/PDF generation, analytics JSON/CSV generation, and raw SQL cross-checks.

VERIFICATION SIGNAL

A report module is only sealed when JSON, PDF, CSV, permissions, pagination, and database cross-truth all pass together.

45. Frontend Dependencies Added

Phase 10 added the dependencies needed for a serious report surface: React Query for server state, TanStack Table for data grids, Recharts for analytics visuals, dayjs for date handling, and Sonner for toast notifications.

These dependencies match the project architecture. They are used for UI state and rendering only; backend remains the source of truth.

Dependency	Purpose
@tanstack/react-query	Server-state fetching, caching, loading/error handling.
@tanstack/react-table	Generic report table rendering and pagination UI.
recharts	Analytics charts G1-G8.
dayjs	Date formatting and filter helpers.
sonner	Toasts for export and error feedback.

46. Root Provider Wiring

src/main.jsx was updated to wire QueryClientProvider and Sonner Toaster at the root. This gives report and chart hooks a consistent cache and gives export actions a clean feedback channel.

This is not a cosmetic change. A reporting page performs multiple parallel requests and needs predictable loading, refetch, and error states.

FRONTEND PRINCIPLE

Server-state belongs in React Query. Local UI choices such as active report, page, and filters belong inside the ReportsLanding orchestrator.

47. ReportsLanding Orchestrator

ReportsLanding.jsx is the /reports page orchestrator. It owns the active report key, shared filters, current page, permission-aware report selection, report query hook, and analytics grid context.

The component follows a strong separation: it orchestrates, but child components render cards, filters, summary tiles, table, export panel, and charts.

State	Meaning
activeKey	The currently selected report. Defaults to first permitted report.
filters	dateFrom, dateTo, divisionId, status. Shared across report/charts where applicable.
page	Current report table page. Resets on filter or report change.
PAGE_SIZE	50 rows per page for UI display.

48. Report Configuration Catalog

reportConfig.js is the frontend report catalog. It defines six report cards, their endpoint keys, labels, permissions, columns, and status enums. This prevents duplicated hardcoding across cards, filters, tables, and export actions.

This design means adding a future report is mostly a catalog addition plus backend endpoint, not a full UI rewrite.

CONFIG PATTERN

Reports should be data-driven where possible: key, label, permission, columns, status options, and endpoint contract.

49. Report Cards Grid

ReportCards.jsx renders the six-card report launcher. It is permission-aware: if the user lacks a report view permission, the card is hidden or unavailable according to the configured gate.

This improves UX while keeping security in the backend. The UI should avoid tempting users with inaccessible actions, but the route must still enforce the rule.

Report card	Permission gate
Calibration Due	reports:view-calibration-due
Pending Jobs	reports:view-pending-jobs
Equipment Utilization	reports:view-equipment-utilization
Engineer Summary	reports:view-engineer-summary
Job Card Summary	reports:view-job-card-summary
Job Request Summary	reports:view-job-request-summary

50. Report Filters

ReportFilters.jsx provides the shared filter bar: date range, division, status, apply, and reset. Status options change based on the active report because not every report uses the same status enum.

This is a good UX pattern: the page looks unified, but report-specific logic still respects each report contract.

- dateFrom and dateTo use YYYY-MM-DD.
- divisionId narrows report and analytics scope.
- status is report-specific.
- Apply triggers refetch through query key changes.
- Reset clears filters and returns page to 1.

51. SummaryTiles Component

SummaryTiles.jsx shows Section-2 KPI tiles for the active report. These values come from backend summaries, not frontend page rows. That distinction matters because a tile should summarize the filtered dataset, not the current page of 50 rows.

Tiles give managers quick status: totals, overdue counts, delayed items, completed counts, utilization percentages, or workload counts depending on active report.

DATA TRUST RULE

A summary tile must represent the full filtered dataset. It must not be computed from visible rows only.

52. ReportTable with TanStack Table

ReportTable.jsx provides a generic table renderer for all six reports. TanStack Table gives column metadata, formatting, pagination integration, and stable rendering without hardcoding six separate table components.

The columns come from reportConfig.js. That gives each report its own field set while keeping the table engine shared.

- Canonical backend fields are displayed directly.
- Status values can be badge-formatted.
- Pagination receives total/page/page_size from backend.
- Sorting and display concerns stay client-side where safe.
- Large exports are handled by PDF endpoints, not by rendering all rows in the browser.

53. ExportPanel

ExportPanel.jsx exposes report export actions: PDF, Excel-like export path, and print. In Phase 10 the authoritative export streams are backend PDF for reports and CSV for charts.

The export button must respect permissions. Users without reports:export should not see or use export actions, and the backend still rejects direct URL access.

Action	Source of truth	Gate
Export PDF	/api/v1/reports/<key>/pdf	report view permission + reports:export
Chart CSV	/api/v1/analytics/<key>/csv	reports:view-analytics + reports:export
Print	Browser print of current view	UI-level action; backend export still gated

54. Analytics Grid

AnalyticsGrid.jsx renders the eight chart cards. Charts fetch in parallel and share filter context where appropriate. This gives a manager a single visual surface for activity, status, workload, divisions, and calibration breakdowns.

Each chart is still independently exportable through its CSV endpoint. That keeps chart visualization and data extraction aligned.

Chart	Title	Endpoint key	Purpose
G1	Monthly Activity	monthly-activity	Time-series view of request/job activity by month.
G2	Equipment Status	equipment-status	Status distribution of equipment records.
G3	Monthly Jobs	monthly-jobs	Job-card creation/completion trend.
G4	Division Wise	division-wise	Division-level distribution of operational work.
G5	Calibration Completion	calibration-completion	Calibration completion trend and throughput.
G6	Job Type Distribution	job-type-distribution	Job request/card mix by work type.
G7	Engineer Workload	engineer-workload	Assigned engineer workload and completion load.
G8	Calibration Status Breakdown	calibration-status-breakdown	Calibration validity/due/overdue breakdown.

55. ChartCard Shell

ChartCard.jsx standardizes chart layout: title, skeleton loading, error state, body, and CSV download icon. This prevents eight chart components from each implementing different loading and export behavior.

In a government-grade UI, consistency matters. A chart that loads differently or exports differently erodes trust.

UI STANDARD

Every chart should show loading, empty, error, and export states consistently. ChartCard is the shell that enforces that discipline.

56. reports.js API Wrapper

src/lib/api/reports.js centralizes fetchReport, downloadReportPdf, fetchChart, and downloadChartCsv. This avoids scattering endpoint strings and blob-download logic across page components.

The wrapper is also the place where axios cancellation, params serialization, and blob response behavior belong.

- fetchReport(key, filters, page, page_size)
- downloadReportPdf(key, filters)
- fetchChart(key, filters)
- downloadChartCsv(key, filters)

57. useReport and useChart Hooks

useReport.js contains React Query hooks for reports and analytics. Query keys encode report key, filters, page, and page size so cache entries are predictable.

This is a major improvement over manual useEffect fetches because React Query provides stale control, loading/error states, refetches, and consistent invalidation behavior.

SERVER STATE RULE

Report rows and chart datasets are server state. They should be fetched and cached through hooks, not manually scattered through components.

58. Route and Sidebar Wiring

App.jsx replaced the /reports placeholder with ReportsLanding. permissions.js changed the Reports nav gate from a broad dashboard permission to reports:view-analytics or report-related access depending on the final nav policy.

This means the sidebar reflects actual Phase 10 capability. Users see Reports when their permissions justify it.

File	Patch meaning
App.jsx	/reports renders ReportsLanding inside protected layout.
permissions.js	Reports nav item becomes permission-aware for Phase 10.
main.jsx	QueryClientProvider and Toaster wired globally.

59. Frontend Build Verification

The final frontend build transformed 2513 modules with no errors. The large bundle warning is expected after adding charts, query, and table libraries; it is a future optimization target, not a correctness failure.

The build passing is necessary but not sufficient. The sealed claim comes from build + smoke + RBAC + export verification together.

BUILD RESULT

FE build clean: 2513 modules transformed, no errors. Future optimization can add dynamic imports/manual chunks if needed.

60. R1 Report Deep Dive — Calibration Due

Calibration Due is exposed as `/api/v1/reports/calibration-due` for JSON and `/api/v1/reports/calibration-due/pdf` for PDF export. Its business purpose: Equipment whose next calibration date is due soon or overdue.

The report reads from `cmms_eqip_mst` + `cmms_section_mst` + last-cal correlated JC subquery. The backend validates filters, queries live data, builds summary tiles, returns paginated canonical rows, and streams PDF on export when the caller also has `reports:export`.

When debugging this report, start at the filter schema, then inspect the repository list query, then verify the summary query uses the same filters, then test the PDF endpoint with a role that has export permission.

REPORT LOGIC

Calibration Due should always keep JSON rows, summary tiles, and PDF rows semantically aligned.

Aspect	Value
JSON endpoint	GET <code>/api/v1/reports/calibration-due</code>
PDF endpoint	GET <code>/api/v1/reports/calibration-due/pdf</code>
View permission	<code>reports:view-calibration-due</code>
Export permission	<code>reports:export</code> additional gate
Primary source	<code>cmms_eqip_mst</code> + <code>cmms_section_mst</code> + last-cal correlated JC subquery

61. R2 Report Deep Dive — Pending Jobs

Pending Jobs is exposed as `/api/v1/reports/pending-jobs` for JSON and `/api/v1/reports/pending-jobs/pdf` for PDF export. Its business purpose: Open job requests not yet closed, with status and assignment visibility.

The report reads from `cmms_jobrequest_mst` + `cmms equip_mst` + `cmms_section_mst`. The backend validates filters, queries live data, builds summary tiles, returns paginated canonical rows, and streams PDF on export when the caller also has `reports:export`.

When debugging this report, start at the filter schema, then inspect the repository list query, then verify the summary query uses the same filters, then test the PDF endpoint with a role that has export permission.

REPORT LOGIC

Pending Jobs should always keep JSON rows, summary tiles, and PDF rows semantically aligned.

Aspect	Value
JSON endpoint	GET /api/v1/reports/pending-jobs
PDF endpoint	GET /api/v1/reports/pending-jobs/pdf
View permission	reports:view-pending-jobs
Export permission	reports:export additional gate
Primary source	cmms_jobrequest_mst + cmms equip_mst + cmms_section_mst

62. R3 Report Deep Dive — Equipment Utilization

Equipment Utilization is exposed as `/api/v1/reports/equipment-utilization` for JSON and `/api/v1/reports/equipment-utilization/pdf` for PDF export. Its business purpose: Equipment-level utilization derived from job-card activity counts.

The report reads from `cmms equip_mst` + `cmms jobcard_mst` aggregates. The backend validates filters, queries live data, builds summary tiles, returns paginated canonical rows, and streams PDF on export when the caller also has `reports:export`.

When debugging this report, start at the filter schema, then inspect the repository list query, then verify the summary query uses the same filters, then test the PDF endpoint with a role that has export permission.

REPORT LOGIC

Equipment Utilization should always keep JSON rows, summary tiles, and PDF rows semantically aligned.

Aspect	Value
JSON endpoint	GET <code>/api/v1/reports/equipment-utilization</code>
PDF endpoint	GET <code>/api/v1/reports/equipment-utilization/pdf</code>
View permission	<code>reports:view-equipment-utilization</code>
Export permission	<code>reports:export</code> additional gate
Primary source	<code>cmms equip_mst</code> + <code>cmms jobcard_mst</code> aggregates

63. R4 Report Deep Dive — Engineer Summary

Engineer Summary is exposed as `/api/v1/reports/engineer-summary` for JSON and `/api/v1/reports/engineer-summary/pdf` for PDF export. Its business purpose: Engineer workload and completion summary by assigned engineer.

The report reads from `cmms_jobcard_mst` grouped by `JM_ASSIGNED_ENGINEER + cmms_emp_mst`. The backend validates filters, queries live data, builds summary tiles, returns paginated canonical rows, and streams PDF on export when the caller also has `reports:export`.

When debugging this report, start at the filter schema, then inspect the repository list query, then verify the summary query uses the same filters, then test the PDF endpoint with a role that has export permission.

REPORT LOGIC

Engineer Summary should always keep JSON rows, summary tiles, and PDF rows semantically aligned.

Aspect	Value
JSON endpoint	<code>GET /api/v1/reports/engineer-summary</code>
PDF endpoint	<code>GET /api/v1/reports/engineer-summary/pdf</code>
View permission	<code>reports:view-engineer-summary</code>
Export permission	<code>reports:export</code> additional gate
Primary source	<code>cmms_jobcard_mst</code> grouped by <code>JM_ASSIGNED_ENGINEER + cmms_emp_mst</code>

64. R5 Report Deep Dive — Job Card Summary

Job Card Summary is exposed as `/api/v1/reports/job-card-summary` for JSON and `/api/v1/reports/job-card-summary/pdf` for PDF export. Its business purpose: Operational job-card list and rollup across card lifecycle states.

The report reads from `cmms_jobcard_mst` + `cmms equip_mst` + `cmms_section_mst` + `employee` joins. The backend validates filters, queries live data, builds summary tiles, returns paginated canonical rows, and streams PDF on export when the caller also has `reports:export`.

When debugging this report, start at the filter schema, then inspect the repository list query, then verify the summary query uses the same filters, then test the PDF endpoint with a role that has export permission.

REPORT LOGIC

Job Card Summary should always keep JSON rows, summary tiles, and PDF rows semantically aligned.

Aspect	Value
JSON endpoint	GET <code>/api/v1/reports/job-card-summary</code>
PDF endpoint	GET <code>/api/v1/reports/job-card-summary/pdf</code>
View permission	<code>reports:view-job-card-summary</code>
Export permission	<code>reports:export</code> additional gate
Primary source	<code>cmms_jobcard_mst</code> + <code>cmms equip_mst</code> + <code>cmms_section_mst</code> + <code>employee</code> joins

65. R6 Report Deep Dive — Job Request Summary

Job Request Summary is exposed as `/api/v1/reports/job-request-summary` for JSON and `/api/v1/reports/job-request-summary/pdf` for PDF export. Its business purpose: Request-level list and status rollout across lifecycle states.

The report reads from `cmms_jobrequest_mst` + equipment and section joins. The backend validates filters, queries live data, builds summary tiles, returns paginated canonical rows, and streams PDF on export when the caller also has `reports:export`.

When debugging this report, start at the filter schema, then inspect the repository list query, then verify the summary query uses the same filters, then test the PDF endpoint with a role that has export permission.

REPORT LOGIC

Job Request Summary should always keep JSON rows, summary tiles, and PDF rows semantically aligned.

Aspect	Value
JSON endpoint	GET /api/v1/reports/job-request-summary
PDF endpoint	GET /api/v1/reports/job-request-summary/pdf
View permission	reports:view-job-request-summary
Export permission	reports:export additional gate
Primary source	cmms_jobrequest_mst + equipment and section joins

66. G1 Analytics Deep Dive — Monthly Activity

Monthly Activity is exposed as `/api/v1/analytics/monthly-activity` for JSON chart data and `/api/v1/analytics/monthly-activity/csv` for CSV export. Purpose: Time-series view of request/job activity by month.

The route is gated by `reports:view-analytics` for JSON. CSV additionally requires `reports:export`. This keeps analytics visible to all allowed roles while restricting downloadable extracts where required.

The chart uses backend aggregation, not mock data. Its CSV export should contain the same dataset currently visualized.

CHART CONTRACT

Monthly Activity must render, refresh, and export from one canonical backend dataset.

Aspect	Value
JSON endpoint	GET <code>/api/v1/analytics/monthly-activity</code>
CSV endpoint	GET <code>/api/v1/analytics/monthly-activity/csv</code>
JSON gate	<code>reports:view-analytics</code>
CSV gate	<code>reports:view-analytics + reports:export</code>
Frontend card	AnalyticsGrid -> ChartCard

67. G2 Analytics Deep Dive — Equipment Status

Equipment Status is exposed as `/api/v1/analytics/equipment-status` for JSON chart data and `/api/v1/analytics/equipment-status/csv` for CSV export. Purpose: Status distribution of equipment records.

The route is gated by `reports:view-analytics` for JSON. CSV additionally requires `reports:export`. This keeps analytics visible to all allowed roles while restricting downloadable extracts where required.

The chart uses backend aggregation, not mock data. Its CSV export should contain the same dataset currently visualized.

CHART CONTRACT
Equipment Status must render, refresh, and export from one canonical backend dataset.

Aspect	Value
JSON endpoint	GET /api/v1/analytics/equipment-status
CSV endpoint	GET /api/v1/analytics/equipment-status/csv
JSON gate	reports:view-analytics
CSV gate	reports:view-analytics + reports:export
Frontend card	AnalyticsGrid -> ChartCard

68. G3 Analytics Deep Dive — Monthly Jobs

Monthly Jobs is exposed as `/api/v1/analytics/monthly-jobs` for JSON chart data and `/api/v1/analytics/monthly-jobs/csv` for CSV export. Purpose: Job-card creation/completion trend.

The route is gated by `reports:view-analytics` for JSON. CSV additionally requires `reports:export`. This keeps analytics visible to all allowed roles while restricting downloadable extracts where required.

The chart uses backend aggregation, not mock data. Its CSV export should contain the same dataset currently visualized.

CHART CONTRACT

Monthly Jobs must render, refresh, and export from one canonical backend dataset.

Aspect	Value
JSON endpoint	GET <code>/api/v1/analytics/monthly-jobs</code>
CSV endpoint	GET <code>/api/v1/analytics/monthly-jobs/csv</code>
JSON gate	<code>reports:view-analytics</code>
CSV gate	<code>reports:view-analytics + reports:export</code>
Frontend card	AnalyticsGrid -> ChartCard

69. G4 Analytics Deep Dive — Division Wise

Division Wise is exposed as `/api/v1/analytics/division-wise` for JSON chart data and `/api/v1/analytics/division-wise/csv` for CSV export. Purpose: Division-level distribution of operational work.

The route is gated by `reports:view-analytics` for JSON. CSV additionally requires `reports:export`. This keeps analytics visible to all allowed roles while restricting downloadable extracts where required.

The chart uses backend aggregation, not mock data. Its CSV export should contain the same dataset currently visualized.

CHART CONTRACT
Division Wise must render, refresh, and export from one canonical backend dataset.

Aspect	Value
JSON endpoint	GET <code>/api/v1/analytics/division-wise</code>
CSV endpoint	GET <code>/api/v1/analytics/division-wise/csv</code>
JSON gate	<code>reports:view-analytics</code>
CSV gate	<code>reports:view-analytics + reports:export</code>
Frontend card	AnalyticsGrid -> ChartCard

70. G5 Analytics Deep Dive — Calibration Completion

Calibration Completion is exposed as `/api/v1/analytics/calibration-completion` for JSON chart data and `/api/v1/analytics/calibration-completion/csv` for CSV export. Purpose: Calibration completion trend and throughput.

The route is gated by `reports:view-analytics` for JSON. CSV additionally requires `reports:export`. This keeps analytics visible to all allowed roles while restricting downloadable extracts where required.

The chart uses backend aggregation, not mock data. Its CSV export should contain the same dataset currently visualized.

CHART CONTRACT

Calibration Completion must render, refresh, and export from one canonical backend dataset.

Aspect	Value
JSON endpoint	GET <code>/api/v1/analytics/calibration-completion</code>
CSV endpoint	GET <code>/api/v1/analytics/calibration-completion/csv</code>
JSON gate	<code>reports:view-analytics</code>
CSV gate	<code>reports:view-analytics + reports:export</code>
Frontend card	AnalyticsGrid -> ChartCard

71. G6 Analytics Deep Dive — Job Type Distribution

Job Type Distribution is exposed as /api/v1/analytics/job-type-distribution for JSON chart data and /api/v1/analytics/job-type-distribution/csv for CSV export. Purpose: Job request/card mix by work type.

The route is gated by reports:view-analytics for JSON. CSV additionally requires reports:export. This keeps analytics visible to all allowed roles while restricting downloadable extracts where required.

The chart uses backend aggregation, not mock data. Its CSV export should contain the same dataset currently visualized.

CHART CONTRACT
Job Type Distribution must render, refresh, and export from one canonical backend dataset.

Aspect	Value
JSON endpoint	GET /api/v1/analytics/job-type-distribution
CSV endpoint	GET /api/v1/analytics/job-type-distribution/csv
JSON gate	reports:view-analytics
CSV gate	reports:view-analytics + reports:export
Frontend card	AnalyticsGrid -> ChartCard

72. G7 Analytics Deep Dive — Engineer Workload

Engineer Workload is exposed as `/api/v1/analytics/engineer-workload` for JSON chart data and `/api/v1/analytics/engineer-workload/csv` for CSV export. Purpose: Assigned engineer workload and completion load.

The route is gated by `reports:view-analytics` for JSON. CSV additionally requires `reports:export`. This keeps analytics visible to all allowed roles while restricting downloadable extracts where required.

The chart uses backend aggregation, not mock data. Its CSV export should contain the same dataset currently visualized.

CHART CONTRACT

Engineer Workload must render, refresh, and export from one canonical backend dataset.

Aspect	Value
JSON endpoint	GET <code>/api/v1/analytics/engineer-workload</code>
CSV endpoint	GET <code>/api/v1/analytics/engineer-workload/csv</code>
JSON gate	<code>reports:view-analytics</code>
CSV gate	<code>reports:view-analytics + reports:export</code>
Frontend card	AnalyticsGrid -> ChartCard

73. G8 Analytics Deep Dive — Calibration Status Breakdown

Calibration Status Breakdown is exposed as `/api/v1/analytics/calibration-status-breakdown` for JSON chart data and `/api/v1/analytics/calibration-status-breakdown/csv` for CSV export. Purpose: Calibration validity/due/overdue breakdown.

The route is gated by `reports:view-analytics` for JSON. CSV additionally requires `reports:export`. This keeps analytics visible to all allowed roles while restricting downloadable extracts where required.

The chart uses backend aggregation, not mock data. Its CSV export should contain the same dataset currently visualized.

CHART CONTRACT

Calibration Status Breakdown must render, refresh, and export from one canonical backend dataset.

Aspect	Value
JSON endpoint	GET <code>/api/v1/analytics/calibration-status-breakdown</code>
CSV endpoint	GET <code>/api/v1/analytics/calibration-status-breakdown/csv</code>
JSON gate	<code>reports:view-analytics</code>
CSV gate	<code>reports:view-analytics</code> + <code>reports:export</code>
Frontend card	AnalyticsGrid -> ChartCard

74. RBAC Philosophy for Phase 10

Reports expose institutional knowledge. In a calibration and maintenance system, knowledge can be sensitive: overdue equipment, engineer workload, pending requests, and job-card completion patterns can reveal operational status. Therefore RBAC is not optional.

Phase 10 applies a two-tier model: view permission controls whether a report or chart can be opened, and export permission controls whether data can be downloaded.

SECURITY PRINCIPLE

Viewing and exporting are separate capabilities. Export creates a portable copy; it must be gated more strongly.

75. Permission Matrix Explained

The permission matrix intentionally varies by role. Super Admin and Lab In-Charge receive broad visibility and export. Lab Engineer gets engineer-relevant views and export. Normal User gets pending/job-request views and export. View-Only gets broad view permissions but no export.

This is more nuanced than a role-only gate and aligns with the sealed project rule: check permission codes, not raw role names, in route logic.

Permission	SA	LIC	LE	NU	VO
reports:view-calibration-due	Y	Y	Y		Y
reports:view-pending-jobs	Y	Y		Y	Y
reports:view-equipment-utilization	Y	Y			Y
reports:view-engineer-summary	Y	Y	Y		Y
reports:view-job-card-summary	Y	Y	Y		Y
reports:view-job-request-summary	Y	Y		Y	Y
reports:view-analytics	Y	Y	Y	Y	Y
reports:export	Y	Y	Y	Y	

76. Composite Export Gate

Composite export gates are the most important RBAC feature in Phase 10. A user may be allowed to view a report but not allowed to export it. This is especially important for View-Only users.

The smoke matrix explicitly proved that VIEW_ONLY receives 200 on /reports/calibration-due but 403 on the PDF endpoint.

User capability	Route result
No token	401 on reports and analytics.
View permission only	200 on JSON, 403 on PDF/CSV.
View permission + reports:export	200 on JSON and export.
Missing report-specific permission	403 even if reports:export exists.

77. Frontend Hiding vs Backend Enforcement

The frontend hides cards and export buttons the user cannot use. This is good UX but not security. The backend route middleware still enforces permissions independently.

A user manually typing a PDF URL must receive 403 if they lack reports:export. This is the difference between a polished UI and a secure system.

DEFENCE IN DEPTH

UI gates reduce confusion. Backend gates enforce truth. Never rely on UI hiding as authorization.

78. No Report Persistence

No report logs and no persisted PDFs means fewer privacy and data-retention concerns. Every export is generated live from the current database state and streamed to the caller.

If future stakeholders request report audit logs, that should become a new sealed phase because it changes data-retention and audit policy.

- No generated PDF rows stored in MySQL.
- No local generated PDF archive.
- No report_log table.
- No background report queue.
- Audit of export events is deferred unless explicitly required.

79. SQL Injection Resistance

Phase 10 uses zod validators and mysql2 placeholders. Status filters are allow-listed enums. Dates are regex-validated. Division IDs are numeric. Column names are not derived from user input.

This is especially important for report modules because they often accept many filters and users may experiment with query strings.

SECURITY RULE

User-controlled values are bound as params. User-controlled column names do not exist. Unknown filters are rejected.

80. Smoke Matrix Overview

Phase 10 was sealed with a 41/41 green smoke matrix. The matrix covers anonymous access, six report JSON/PDF flows, eight analytics JSON/CSV flows, pagination, RBAC, cross-truth raw SQL checks, and filter narrowing.

This is the correct standard because reporting bugs are not only compile errors. They are wrong totals, wrong permissions, broken exports, or inconsistent filters.

Category	Evidence
Authentication	Anonymous requests to reports/analytics return 401.
Report JSON/PDF	R1-R6 each pass JSON, PDF, pagination.
Analytics JSON/CSV	G1-G8 each pass JSON and CSV.
RBAC	VIEW_ONLY can view allowed JSON but cannot export PDF.
Cross-truth	Report totals match raw COUNT queries.
Frontend	Production build clean.

81. Smoke Evidence — X1 anonymous -> 401 on /reports + /analytics

Test evidence: X1 anonymous -> 401 on /reports + /analytics.

This check exists because a reporting module can appear visually correct while still failing authorization, pagination, export streaming, or database consistency. The smoke matrix turns each sealed claim into evidence.

When future changes touch reports, analytics, permissions, or database aliases, this check should be re-run or represented in the next smoke suite.

SMOKE CHECK

X1 anonymous -> 401 on /reports + /analytics

82. Smoke Evidence — R1 Calibration Due JSON + PDF + pagination, 5704 rows,

Test evidence: R1 Calibration Due JSON + PDF + pagination, 5704 rows, about 1.6 MB PDF.

This check exists because a reporting module can appear visually correct while still failing authorization, pagination, export streaming, or database consistency. The smoke matrix turns each sealed claim into evidence.

When future changes touch reports, analytics, permissions, or database aliases, this check should be re-run or represented in the next smoke suite.

SMOKE CHECK

R1 Calibration Due JSON + PDF + pagination, 5704 rows, about 1.6 MB PDF

83. Smoke Evidence — R2 Pending Jobs JSON + PDF + pagination, 21493 rows, ab

Test evidence: R2 Pending Jobs JSON + PDF + pagination, 21493 rows, about 2.9 MB PDF.

This check exists because a reporting module can appear visually correct while still failing authorization, pagination, export streaming, or database consistency. The smoke matrix turns each sealed claim into evidence.

When future changes touch reports, analytics, permissions, or database aliases, this check should be re-run or represented in the next smoke suite.

SMOKE CHECK

R2 Pending Jobs JSON + PDF + pagination, 21493 rows, about 2.9 MB PDF

84. Smoke Evidence — R3 Equipment Utilization JSON + PDF + pagination, 5705

Test evidence: R3 Equipment Utilization JSON + PDF + pagination, 5705 rows, about 870 KB PDF.

This check exists because a reporting module can appear visually correct while still failing authorization, pagination, export streaming, or database consistency. The smoke matrix turns each sealed claim into evidence.

When future changes touch reports, analytics, permissions, or database aliases, this check should be re-run or represented in the next smoke suite.

SMOKE CHECK

R3 Equipment Utilization JSON + PDF + pagination, 5705 rows, about 870 KB PDF

85. Smoke Evidence — R4 Engineer Summary JSON + PDF + pagination, 1 row, abo

Test evidence: R4 Engineer Summary JSON + PDF + pagination, 1 row, about 4.8 KB PDF.

This check exists because a reporting module can appear visually correct while still failing authorization, pagination, export streaming, or database consistency. The smoke matrix turns each sealed claim into evidence.

When future changes touch reports, analytics, permissions, or database aliases, this check should be re-run or represented in the next smoke suite.

SMOKE CHECK

R4 Engineer Summary JSON + PDF + pagination, 1 row, about 4.8 KB PDF

86. Smoke Evidence — R5 Job Card Summary JSON + PDF + pagination, 19438 rows

Test evidence: R5 Job Card Summary JSON + PDF + pagination, 19438 rows, about 2.9 MB PDF.

This check exists because a reporting module can appear visually correct while still failing authorization, pagination, export streaming, or database consistency. The smoke matrix turns each sealed claim into evidence.

When future changes touch reports, analytics, permissions, or database aliases, this check should be re-run or represented in the next smoke suite.

SMOKE CHECK

R5 Job Card Summary JSON + PDF + pagination, 19438 rows, about 2.9 MB PDF

87. Smoke Evidence — R6 Job Request Summary JSON + PDF + pagination, 21513 r

Test evidence: R6 Job Request Summary JSON + PDF + pagination, 21513 rows, about 2.9 MB PDF.

This check exists because a reporting module can appear visually correct while still failing authorization, pagination, export streaming, or database consistency. The smoke matrix turns each sealed claim into evidence.

When future changes touch reports, analytics, permissions, or database aliases, this check should be re-run or represented in the next smoke suite.

SMOKE CHECK

R6 Job Request Summary JSON + PDF + pagination, 21513 rows, about 2.9 MB PDF

88. Smoke Evidence — Cross-truth R3 total_equipment == COUNT(cmms equip_mst)

Test evidence: Cross-truth R3 total_equipment == COUNT(cmms equip_mst) -> 5705 == 5705.

This check exists because a reporting module can appear visually correct while still failing authorization, pagination, export streaming, or database consistency. The smoke matrix turns each sealed claim into evidence.

When future changes touch reports, analytics, permissions, or database aliases, this check should be re-run or represented in the next smoke suite.

SMOKE CHECK

Cross-truth R3 total_equipment == COUNT(cmms equip_mst) -> 5705 == 5705

89. Smoke Evidence — Cross-truth R2 total_pending == COUNT(pending statuses)

Test evidence: Cross-truth R2 total_pending == COUNT(pending statuses) -> 21493 == 21493.

This check exists because a reporting module can appear visually correct while still failing authorization, pagination, export streaming, or database consistency. The smoke matrix turns each sealed claim into evidence.

When future changes touch reports, analytics, permissions, or database aliases, this check should be re-run or represented in the next smoke suite.

SMOKE CHECK

Cross-truth R2 total_pending == COUNT(pending statuses) -> 21493 == 21493

90. Smoke Evidence — Filter smoke R6 status=VERIFIED_CLOSED narrowed correct

Test evidence: Filter smoke R6 status=VERIFIED_CLOSED narrowed correctly -> 0 == 0.

This check exists because a reporting module can appear visually correct while still failing authorization, pagination, export streaming, or database consistency. The smoke matrix turns each sealed claim into evidence.

When future changes touch reports, analytics, permissions, or database aliases, this check should be re-run or represented in the next smoke suite.

SMOKE CHECK

Filter smoke R6 status=VERIFIED_CLOSED narrowed correctly -> 0 == 0

91. Smoke Evidence — G1..G8 JSON + CSV passed for all 8 charts

Test evidence: G1..G8 JSON + CSV passed for all 8 charts.

This check exists because a reporting module can appear visually correct while still failing authorization, pagination, export streaming, or database consistency. The smoke matrix turns each sealed claim into evidence.

When future changes touch reports, analytics, permissions, or database aliases, this check should be re-run or represented in the next smoke suite.

SMOKE CHECK

G1..G8 JSON + CSV passed for all 8 charts

92. Smoke Evidence — VIEW_ONLY 200 on /reports/calibration-due, 403 on /pdf

Test evidence: VIEW_ONLY 200 on /reports/calibration-due, 403 on /pdf export gate.

This check exists because a reporting module can appear visually correct while still failing authorization, pagination, export streaming, or database consistency. The smoke matrix turns each sealed claim into evidence.

When future changes touch reports, analytics, permissions, or database aliases, this check should be re-run or represented in the next smoke suite.

SMOKE CHECK

VIEW_ONLY 200 on /reports/calibration-due, 403 on /pdf export gate

93. Smoke Evidence — Frontend build clean: 2513 modules transformed, no error

Test evidence: Frontend build clean: 2513 modules transformed, no errors.

This check exists because a reporting module can appear visually correct while still failing authorization, pagination, export streaming, or database consistency. The smoke matrix turns each sealed claim into evidence.

When future changes touch reports, analytics, permissions, or database aliases, this check should be re-run or represented in the next smoke suite.

SMOKE CHECK

Frontend build clean: 2513 modules transformed, no errors

94. Cross-Truth Verification

Cross-truth tests compare report totals with raw SQL counts. This is stronger than simply checking that the endpoint returns 200. It proves the report math agrees with database truth.

Phase 10 cross-truth examples include Equipment Utilization total_equipment matching COUNT(cmms equip_mst) and Pending Jobs total_pending matching the raw pending-status count.

Check	Result
R3 total_equipment == COUNT(cmms equip_mst)	5705 == 5705
R2 total_pending == COUNT(pending statuses)	21493 == 21493
R6 status=VERIFIED_CLOSED filter narrowing	0 == 0 in legacy data

95. PDF Stream Verification

PDF tests verify more than file creation. They confirm endpoint access, export permission, response streaming, and the renderer handling thousands of rows without crashing.

The smoke results show large PDF outputs for report-heavy tables, proving that auto-pagination and streaming are not only theoretical.

Report	PDF evidence
R1 Calibration Due	5704 rows, about 1.6 MB PDF
R2 Pending Jobs	21493 rows, about 2.9 MB PDF
R3 Equipment Utilization	5705 rows, about 870 KB PDF
R4 Engineer Summary	1 row, about 4.8 KB PDF
R5 Job Card Summary	19438 rows, about 2.9 MB PDF
R6 Job Request Summary	21513 rows, about 2.9 MB PDF

96. CSV Verification

Every analytics chart has a CSV export endpoint. The smoke matrix verified JSON and CSV for all eight charts. This proves that chart data is not trapped in the UI and can be exported for spreadsheet analysis.

CSV export is intentionally chart-specific; there is no monolithic export endpoint. This keeps each dataset small, relevant, and easy to validate.

CSV RESULT

G1 through G8 JSON + CSV passed.

97. Frontend Production Build Verification

The frontend build passed with 2513 modules transformed and no errors. The chunk-size warning is not a failure. It is a future optimization note after adding Recharts, React Query, TanStack Table, dayjs, and Sonner.

A future phase can code-split /reports or configure manual chunks. Phase 10 correctness is green.

BUILD SEAL

Production build clean. Large chunk warning acknowledged as performance backlog, not a functional blocker.

98. How to Use Phase 10 as a User

A permitted user opens /reports, selects a report card, applies filters, reads summary tiles, inspects table rows, and exports PDF if they hold reports:export. The same page shows analytics charts and permits CSV export for users with export permission.

The page is designed for technical staff: minimal ceremony, fast filtering, clear summaries, and downloadable proof when needed.

- Open Reports from the sidebar.
- Select Calibration Due, Pending Jobs, Equipment Utilization, Engineer Summary, Job Card Summary, or Job Request Summary.
- Use date/division/status filters.
- Review summary tiles and detailed table.
- Export PDF when permitted.
- Review analytics charts and export CSV when permitted.

99. How to Debug a Wrong Report Number

Start by identifying whether the problem is row selection, summary aggregation, permission scope, or frontend display. Do not immediately change the UI.

The correct debug path is: reproduce filters, inspect API response, compare to raw SQL, inspect repo query, inspect service summary, then inspect frontend table/tiles.

Step	Action
1	Capture report key and filters from URL/network request.
2	Call JSON endpoint directly with same token.
3	Compare total/summary against raw SQL count.
4	Inspect reports.repo.js WHERE builder and joins.
5	Inspect reports.service.js summary assembly.
6	Inspect reportConfig columns and SummaryTiles rendering.

100. How to Debug a PDF Export Bug

A PDF issue can be authorization, data, layout, or streaming. Separate those concerns before editing reports.pdf.js.

First verify the user has reports:export. Then call the JSON endpoint. If JSON is correct but PDF is wrong, the bug is in rendering. If both are wrong, the bug is in repo/service logic.

PDF DEBUG RULE

Do not debug PDF layout before proving the JSON dataset is correct. PDF should render the same report contract.

101. How to Add a Future Report

Future reports should follow the Phase 10 pattern instead of copying and hacking UI code. A new report requires a permission code, validator schema, repo list/summary functions, service registration, route entries, PDF section renderer, frontend reportConfig entry, and smoke tests.

The key is to add the report through the pipeline, not as a one-off UI screen. If the report cannot pass JSON, PDF, RBAC, pagination, and cross-truth checks, it is not sealed.

FUTURE REPORT RULE

A new report is a complete vertical slice: permission, validator, repo, service, routes, frontend config, PDF renderer, and smoke evidence.

102. Future Report Addition Checklist

Use this checklist when Phase 11 or later adds a new report. Every item matters because it keeps frontend, backend, export, and RBAC aligned.

- Define business question and report owner.
- Define canonical columns and summary tiles.
- Map canonical fields to real DB columns in repo only.
- Add zod query schema with strict() unknown-param rejection.
- Add list/count/summary queries with bound params.
- Add route JSON and PDF endpoints.
- Add permission code and role grants through additive migration.
- Add frontend reportConfig entry and table columns.
- Add PDF section renderer and smoke tests.
- Verify cross-truth with raw SQL counts.

103. Operational Monitoring After Phase 10

After deployment, monitoring should focus on response time, PDF generation failures, 403 rate, invalid filter 422s, and query performance on large date windows.

Large report PDFs are expected to take more time than JSON. However, they should not crash the process or consume excessive memory.

Signal	Healthy meaning	Action if abnormal
PDF endpoint failures	Rare; mostly permission-related	Check PDFKit rendering and large-row pagination.
403 spikes	Users trying inaccessible exports	Check sidebar/export button hiding and permissions.
422 spikes	Bad filter UI or manual URL misuse	Check ReportFilters and validator contract.
Slow JSON reports	Index/filter problem	EXPLAIN repo query and confirm WHERE fields.
Slow PDFs	Very large row set	Consider export page limit or queued export in future phase.

104. Deployment Notes

Phase 10 deployment requires backend dependencies installed, frontend dependencies installed, migration 400 applied, backend restarted with /reports and /analytics mounted, and frontend rebuilt with ReportsLanding route active.

Because migration 400 is permission-only and idempotent, it is low risk. Still, it must be run before login tokens are refreshed, otherwise users may not receive new report permissions until token refresh/re-login.

- Run backend npm install for pdfkit.
- Run frontend npm install for React Query, TanStack Table, Recharts, dayjs, Sonner.
- Apply migration 400.
- Restart backend.
- Rebuild and deploy frontend.
- Ask test users to re-login or refresh token so new permissions are present.
- Run smoke_phase10.js against target environment.

105. Appendix A — Endpoint Matrix

This appendix records the complete Phase 10 endpoint surface. Reports expose JSON and PDF endpoints; analytics expose JSON and CSV endpoints.

Type	Routes
Reports JSON (6)	/api/v1/reports/{calibration-due,pending-jobs,equipment-utilization,engineer-summary,job-card-summary,job-request-summary}
Reports PDF (6)	Same report paths + /pdf
Analytics JSON (8)	/api/v1/analytics/{monthly-activity,equipment-status,monthly-jobs,division-wise,calibration-completion,job-type-distribution,engineer-workload,calibration-status-breakdown}
Analytics CSV (8)	Same analytics paths + /csv

106. Appendix B — Backend File Inventory

The backend delivered 15 major files or file-level changes. This appendix is the handover inventory for future debugging and code review.

- db/migrations/400__phase10_reports_permissions.sql
- src/modules/reports/reports.validators.js
- src/modules/reports/reports.repo.js
- src/modules/reports/reports.service.js
- src/modules/reports/reports.pdf.js
- src/modules/reports/reports.controller.js
- src/modules/reports/reports.routes.js
- src/modules/analytics/analytics.validators.js
- src/modules/analytics/analytics.repo.js
- src/modules/analytics/analytics.service.js
- src/modules/analytics/analytics.controller.js
- src/modules/analytics/analytics.routes.js
- src/server.js mount patch
- db/discovery/smoke_phase10.js
- BE dependency: pdffkit

107. Appendix C — Frontend File Inventory

The frontend delivered report page components, chart components, API wrappers, hooks, root provider patches, routing patches, and permission navigation changes.

- `src/pages/reports/reportConfig.js`
- `src/pages/reports/ReportsLanding.jsx`
- `src/pages/reports/ReportCards.jsx`
- `src/pages/reports/ReportFilters.jsx`
- `src/pages/reports/SummaryTiles.jsx`
- `src/pages/reports/ReportTable.jsx`
- `src/pages/reports/ExportPanel.jsx`
- `src/pages/reports/charts/chartTheme.js`
- `src/pages/reports/charts/ChartCard.jsx`
- `src/pages/reports/charts/AnalyticsGrid.jsx`
- `src/lib/api/reports.js`
- `src/lib/hooks/useReport.js`
- `src/main.jsx` QueryClientProvider + Toaster
- `src/App.jsx` Reports route
- `src/lib/permissions.js` nav gate patch
- FE deps: `@tanstack/react-query`, `@tanstack/react-table`, `recharts`, `dayjs`, `sonner`

108. Appendix D — Reports Catalog

This is the final report catalog in one table. It is the reference for UX, API, permissions, and QA naming.

ID	Report	Key	Primary source
R1	Calibration Due	calibration-due	cmms equip_mst + cmms_section_mst + last-cal correlated JC subquery
R2	Pending Jobs	pending-jobs	cmms_jobrequest_mst + cmms equip_mst + cmms_section_mst
R3	Equipment Utilization	equipment-utilization	cmms equip_mst + cmms_jobcard_mst aggregates
R4	Engineer Summary	engineer-summary	cmms_jobcard_mst grouped by JM_ASSIGNED_ENGINEER + cmms_emp_mst
R5	Job Card Summary	job-card-summary	cmms_jobcard_mst + cmms equip_mst + cmms_section_mst + employee joins
R6	Job Request Summary	job-request-summary	cmms_jobrequest_mst + equipment and section joins

109. Appendix E — Analytics Catalog

This is the final analytics chart catalog. Each item has JSON and CSV endpoints.

ID	Chart	Key	Purpose
G1	Monthly Activity	monthly-activity	Time-series view of request/job activity by month.
G2	Equipment Status	equipment-status	Status distribution of equipment records.
G3	Monthly Jobs	monthly-jobs	Job-card creation/completion trend.
G4	Division Wise	division-wise	Division-level distribution of operational work.
G5	Calibration Completion	calibration-completion	Calibration completion trend and throughput.
G6	Job Type Distribution	job-type-distribution	Job request/card mix by work type.
G7	Engineer Workload	engineer-workload	Assigned engineer workload and completion load.
G8	Calibration Status Breakdown	calibration-status-breakdown	Calibration validity/due/overdue breakdown.

110. Appendix F — Filter Contract

Filter contracts are deliberately conservative. They are strict, validated, and consistent with report-specific needs.

Filter	Type	Notes
dateFrom	YYYY-MM-DD	Inclusive start of day.
dateTo	YYYY-MM-DD	Inclusive end of day.
divisionId	positive integer	Maps to cmms_section_mst.SM_ID relationship.
status	allow-listed enum	Different per report.
page	integer >= 1	Defaults to 1.
page_size	1..500	Defaults to 50.

111. Appendix G — Smoke Matrix

This is the smoke evidence condensed for handover. The source transcript records 41 passed and 0 failed.

- X1 anonymous -> 401 on /reports + /analytics
- R1 Calibration Due JSON + PDF + pagination, 5704 rows, about 1.6 MB PDF
- R2 Pending Jobs JSON + PDF + pagination, 21493 rows, about 2.9 MB PDF
- R3 Equipment Utilization JSON + PDF + pagination, 5705 rows, about 870 KB PDF
- R4 Engineer Summary JSON + PDF + pagination, 1 row, about 4.8 KB PDF
- R5 Job Card Summary JSON + PDF + pagination, 19438 rows, about 2.9 MB PDF
- R6 Job Request Summary JSON + PDF + pagination, 21513 rows, about 2.9 MB PDF
- Cross-truth R3 total_equipment == COUNT(cmms equip_mst) -> 5705 == 5705
- Cross-truth R2 total_pending == COUNT(pending statuses) -> 21493 == 21493
- Filter smoke R6 status=VERIFIED_CLOSED narrowed correctly -> 0 == 0
- G1..G8 JSON + CSV passed for all 8 charts
- VIEW_ONLY 200 on /reports/calibration-due, 403 on /pdf export gate
- Frontend build clean: 2513 modules transformed, no errors

112. Appendix H — Glossary

The vocabulary below should be used consistently in code reviews and documentation.

Term	Meaning
Report	Operational row-level view with summary and PDF export.
Analytics	Aggregated chart dataset with CSV export.
Canonical field	Frontend/service-friendly field name independent of legacy DB naming.
Repository aliasing	Mapping EQM/JR/JM columns into canonical names in SQL SELECT.
Composite export gate	View permission plus reports:export.
Cross-truth check	Smoke test comparing endpoint totals against raw SQL counts.
Streamed PDF	Generated and sent to response without persisting a file record.

113. Appendix I — Senior Review Checklist

Use this checklist before calling any future reports patch sealed.

- Does every route have authenticate and authorize?
- Does every export route require reports:export?
- Are all filters validated with strict zod schemas?
- Are all SQL values bound through placeholders?
- Are legacy column names isolated inside repositories?
- Do summary totals match filtered dataset, not page rows?
- Do JSON and PDF outputs use the same report semantics?
- Do chart JSON and CSV outputs match?
- Does frontend hide inaccessible cards/actions?
- Does backend still enforce the same access if URL is typed manually?
- Does smoke cover 401, 403, JSON, PDF, CSV, pagination, and cross-truth?

114. Appendix J — Final Seal Statement

Phase 10 is sealed as the Reports & Analytics phase. It gives CMCMIS the institutional memory layer: live reports, printable PDFs, visual analytics, CSV extracts, and permission-safe access control.

The biggest engineering value is not the number of screens; it is the discipline. Reports read live data, exports are gated, legacy names are contained, PDFs are streamed, charts are backed by SQL, and the smoke matrix proves the claims.

This document should be kept with the Phase 0-9 sealed documents and used as the authoritative Phase 10 build record unless a later sealed phase explicitly supersedes it.

SEALED

Phase 10 — Reports & Analytics — fully documented, DS style, senior-engineering ready, 41/41 smoke green.

115. Appendix K1 — Reviewer Notes Page 1

This page is intentionally available for DS / reviewer notes during handover. Keep any manual review findings here rather than modifying sealed logic sections without a version bump.

If a note leads to a code change, create a new patch document or a Phase 10.1 addendum with explicit smoke evidence.

NOTE DISCIPLINE

Do not silently edit sealed decisions. Record review findings, patch intentionally, re-render, and re-smoke.

116. Appendix K2 — Reviewer Notes Page 2

This page is intentionally available for DS / reviewer notes during handover. Keep any manual review findings here rather than modifying sealed logic sections without a version bump.

If a note leads to a code change, create a new patch document or a Phase 10.1 addendum with explicit smoke evidence.

NOTE DISCIPLINE

Do not silently edit sealed decisions. Record review findings, patch intentionally, re-render, and re-smoke.

117. Appendix K3 — Reviewer Notes Page 3

This page is intentionally available for DS / reviewer notes during handover. Keep any manual review findings here rather than modifying sealed logic sections without a version bump.

If a note leads to a code change, create a new patch document or a Phase 10.1 addendum with explicit smoke evidence.

NOTE DISCIPLINE

Do not silently edit sealed decisions. Record review findings, patch intentionally, re-render, and re-smoke.

118. Appendix K4 — Reviewer Notes Page 4

This page is intentionally available for DS / reviewer notes during handover. Keep any manual review findings here rather than modifying sealed logic sections without a version bump.

If a note leads to a code change, create a new patch document or a Phase 10.1 addendum with explicit smoke evidence.

NOTE DISCIPLINE

Do not silently edit sealed decisions. Record review findings, patch intentionally, re-render, and re-smoke.

